

MATH 302 Snippets

The operations the lecture notes type most, and the snippet for each

These come from counting every construct inside `$ ... $` in `L01.md`–`L10.md`, then asking which ones cost real keystrokes: the padded set-builder, balls, the interior and closure, the two metric spaces every example lives in, and the inequalities that get rewritten in every proof. The operations that were common but already cost nothing (native Typst, or a snippet you already had) are in Section 5, so the list here stays short.

All of them are <Tab> snippets. Every trigger is name-shaped, and the rule in `snippetsGuide.md` holds: no autosnippet shadows something you might mean literally. They fire only inside math, in `.typ` files and in markdown notes, except the three marked `MD`, which exist only in the notes (see Section 6).

Reading the tables. `c` is a tab stop: its text is the default, so type over it or <Tab> past it; `...` is an empty one. `c` repeats an earlier stop and changes as you type there. The Renders column fills empty stops with an example.

1. The topology of a set

Trigger	Expands to	Renders	Uses
<code>intr MD</code>	<code>intr(...)</code>	A°	224
<code>cl MD</code>	<code>cl(...)</code>	$\overline{A}$	110
<code>bdy</code>	<code>"Bdy"(...)</code>	$\text{Bdy}(A)$	8

The most-typed things in the course. All three take a visual selection: select `A union B`, press <Tab> (it disappears), type `cl<Tab>`, and it comes back as `cl(A union B)`. The complement needs no snippet, `A^c` is three keys.

2. Balls

Trigger	Expands to	Renders	Uses
<code>ball</code>	<code>B(c, r)</code>	$B(c, r)$	122
<code>cball</code>	<code>B[c, r]</code>	$B[c, r]$	22
<code>sph</code>	<code>S(c, r)</code>	$S(c, r)$	13
<code>bsub</code>	<code>B(c, r) subset.eq A</code>	$B(c, r) \subseteq A$	40
<code>ipt</code>	<code>exists thin r > 0 "such that" B(c, r) subset.eq A</code>	$\exists r > 0 \text{ such that } B(c, r) \subseteq A$	8

`ipt` is the interior-point condition, the one every “is open” proof starts from. Type the radius once (`epsilon`, `r_1`) and the ball picks it up.

3. Sets

Trigger	Expands to	Renders	Uses
<code>sb</code>	<code>{ thin x in M med bar.v med ... thin }</code>	$\{ x \in M \mid d(x, c) < r \}$	29
<code>bcup</code>	<code>union.big_(i in I) A_i</code>	$\bigcup_{i \in I} A_i$	23
<code>bcap</code>	<code>inter.big_(i in I) A_i</code>	$\bigcap_{i \in I} A_i$	13
<code>lst</code>	<code>x_1, dots.h, x_n</code>	$x_1, \dots, x_n$	7
<code>tup</code>	<code>(x_1, dots.h, x_n)</code>	$(x_1, \dots, x_n)$	7

`sb` writes the set-builder with the spacing the notes use, the part that takes twenty keystrokes to type by hand. It is not `set`, which is `#set` in a `.typ` file; folio’s `setb` gives the colon form instead. `bcup` and `bcap` borrow the LaTeX names. For $\bigcup_{n=1}^\infty$, type `n = 1` over the index and add `^oo` after the parenthesis.

4. Metrics and inequalities

Trigger	Expands to	Renders	Uses
usual	(<code>RR</code> , "usual")	$(\mathbb{R}, \text{usual})$	37
disc	(<code>M</code> , "discrete")	$(M, \text{discrete})$	13
tri	<code>d(a, b) <= d(a, c) + d(c, b)</code>	$d(a, b) \leq d(a, c) + d(c, b)$	8
ntri	<code>norm(x + y) <= norm(x) + norm(y)</code>	$\ x + y\ \leq \ x\ + \ y\ $	5
ip <code>MD</code>	<code>ip(x, y)</code>	$\langle x, y \rangle$	103

`tri` asks for three points and writes all six slots in the right order, which is exactly where a hand-typed triangle inequality goes wrong. The same with `ntri` and two vectors.

5. Common, but no snippet needed

Typst or an existing snippet already types these in a few keys. The ones on the left are what the notes write, some of them the long way.

In the notes	Uses	Type instead
<code>bb(R)</code> , <code>bb(Q)</code> , ...	343	<code>RR QQ ZZ CC NN</code> : native, and <code>RR^n</code> for $\mathbb{R}^n$
<code>norm(...)</code> , <code>abs(...)</code>	282	the names themselves, or the existing <code>norm</code> , <code>abs</code> snippets
<code>d(x, y)</code>	187	just type it; <code>tri</code> covers the triangle inequality
<code>A^c</code>	164	just type it
<code>subset.eq</code>	160	<code>sub=</code> (auto) for $\subseteq$, <code>sup=</code> for $\supseteq$
med thin quad	322	just type them; <code>sb</code> writes the set-builder's for you
"text"	202	<code>qq</code> (auto) gives " <code>...</code> "
nothing	81	<code>;0</code> (auto)
lambda epsilon tau	153	<code>;l ;e ;u</code> (auto)
<code><= ==></code> <code><==></code> <code>:= != >=</code>	212	native shorthands, no snippet
union inter	77	native names; <code>bcup bcap</code> for the indexed ones
<code>dot.op</code>	44	<code>dot</code> alone is $\cdot$, so <code>norm(dot)</code> is $\ \cdot\ $
<code>1 slash n</code>	27	<code>1 \ / n</code> : the escaped slash, as L01 already does
<code>dots.h</code> , <code>dots</code>	28	<code>...</code> is <code>...</code> , a native shorthand
<code>sqrt</code> , underbrace	49	the existing <code>sqrt</code> and <code>ub</code> snippets
<code>in.not</code>	11	<code>!in</code> (auto)
<code>(M, d)</code>	35	just type it; <code>usual</code> and <code>disc</code> cover the two named spaces

6. Where they work

- In a markdown note every snippet above works, inside `$ ... $` or `$$ ... $$`. `intr`, `cl` and `ip` are macros of the vault's TypstMate preamble (`.obsidian/plugins/typst-mate/data.json`), so they live in `markdown.lua` and exist only there.
- In a `.typ` everything except the three `MD` ones. There `ip` is folio's (from `typst.lua`, defaulting to `u`, `v`); `intr` and `cl` need their `#lets`, which `textbook.typ` has at the top.
- After `:MdMath latex` none of these fire: the buffer takes `mathmode.lua`'s LaTeX snippets instead, and the three preamble ones check the mode.